TokenEx Setup Guide

What You Need

To enable live transactions through the Digitzs MID with TokenEx, you need two credentials from your TokenEx account:

1. **TokenEx ID** - Already configured: `6747114507881848`
2. **Client Secret Key** - This is what you need to find/configure

Where to Find Your TokenEx Credentials

Option 1: TokenEx Client Portal

1. Log into your TokenEx account at: https://portal.tokenex.com
2. Navigate to **Account Settings** or **API Settings**
3. Look for:
   - **TokenEx ID**: Should show `6747114507881848`
   - **Client Secret Key**: A long alphanumeric string (e.g., `A1b2C3D4e5F6h7I8j9K0l1M2n3O4p`)

Option 2: AWS Secrets Manager (If Managed via AWS)

If your TokenEx credentials are stored in AWS Secrets Manager:

1. Log into AWS Console

2. Navigate to **AWS Secrets Manager**
3. Search for secrets containing "tokenex" or "payment"
4. Look for keys named:
    - `TOKENEX_ID`
    - `TOKENEX_SECRET_KEY` or `TOKENEX_CLIENT_SECRET`

Option 3: Contact TokenEx Support

If you can't locate your credentials:

- Email: support@tokenex.com
- They can regenerate your Client Secret Key if needed

How the Integration Works

The checkout now uses a **secure server-side authentication flow**:

1. **Frontend requests auth** → Calls edge function `/functions/v1/tokenex-auth`
2. **Edge function generates HMAC** → Uses Client Secret Key (kept secure server-side)
3. **Returns authentication key** → Valid for 20 minutes, includes timestamp
4. **TokenEx iframe loads** → With dynamically generated auth key
5. **Card data tokenized** → At device level, never touches our servers
6. **Payment processed** → Through Digitzs MID → ProPay → Stripe

Current Status

Demo Mode Active:

- TokenEx ID is configured

- Client Secret Key needs to be added as environment variable
- Currently shows mock transactions without the secret key

To Enable Live Transactions:

1. Get your Client Secret Key from TokenEx
2. Add it as a Supabase secret (automatically configured)
3. Transactions will route through:
    - **TokenEx** (tokenization layer)
    - **Digitzs MID**: `digitzs-escapefrom-33738480-5013250-1771270463`
    - **ProPay MID**: `33738480-5013250-1771270463`
    - **Stripe** (processor)

Authentication Key Generation

The edge function at `/supabase/functions/tokenex-auth/index.ts` generates the authentication key using:

```
HMAC-SHA256(tokenExID|origin|timestamp|tokenScheme, clientSecretK
```

This is Base64 encoded and returned to the frontend, where it's used to initialize the TokenEx iframe.

The authentication key expires after 20 minutes for security purposes.

Testing With Live Credentials

Once you add the `TOKENEX_SECRET_KEY`:

1. The iframe will load with real tokenization
2. Test cards will be tokenized at device level

3. You can run $1 parking pass transactions to test
4. Full transaction flow will execute through Digitzs → Stripe

Parking Pass vs Admission

The checkout now supports two ticket types:

- **General Admission**: $32/person
- **Parking Pass**: $1/vehicle (perfect for testing live transactions)

Both use the same payment flow through TokenEx + Digitzs MID.

API Documentation Reference

The TokenEx authentication follows their official spec: https://documentation.ixopay.com/modules/docs/tokenex/generating-the-authentication-key

Key requirements:

- HMAC-SHA256 algorithm
- Base64 encoding
- Timestamp in `yyyyMMddHHmmss` UTC format
- 20-minute expiration window
- HTTPS required in production